INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Excelencia en Educación Superior

1.1 Neurona Perceptron

Erick Mejia

Caso Práctico: Neurona de McCulloch y Pitts

2. Aplicando la MPNeurona a un caso práctico real

2.1. Conjunto de datos

Esta es una copia del conjunto de datos de UCI ML Breast Cancer Wisconsin.

Descripción de la muestra

Las características de entrada se calculan a partir de una imagen digitalizada de un aspirado de aguja fina (FNA) de una masa mamaria. Describen las características de los núcleos celulares presentes en la imagen.

El plan de separación descrito anteriormente se obtuvo utilizando el método de árbol de múltiples superficies (MSM-T) "Construcción de un árbol de decisión mediante programación lineal".

El programa lineal utilizado para obtener el plano de separación en el espacio tridimensional es el que describe en [K. P. Bennett y O. L. Mangasarian].

Lectura de datos

In [1]:
```
from sklearn.datasets import load_breast_cancer

cancer_mama = load_breast_cancer()

# Separamos los datos de entrada y los datos de salida
X = cancer_mama.data
Y = cancer_mama.target
```

In [2]:
```
dir(cancer_mama)
```

Out[2]:
```
['DESCR',
 'data',
 'data_module',
 'feature_names',
 'filename',
 'frame',
 'target',
 'target_names']
```

In [3]:
```
print(X)
```

```
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
```

Visualización del conjunto de datos

In [4]:
```
import pandas as pd

df = pd.DataFrame(X, columns=cancer_mama.feature_names)
df
```

Out[4]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	sy
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	
...	...	...	...	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	

569 rows × 30 columns

In [5]:
```
print(Y)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

División del conjunto de datos

In [6]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(df, Y, stratify=Y)

print("Tamaño de entrenamiento ", len(X_train))
print("Tamalo de pruebas", len(X_test))
```

```
Tamaño de entrenamiento  426
Tamalo de pruebas 143
```

Implementación de un aMP Neurona más avanzada

In [7]:
```python
# importamos la biblioteca de numpy
import numpy as np
from sklearn.metrics import accuracy_score

class MpNeurona:
    # creamos nuestro constructor
    def __init__(self):
        self.threshold = None

    # Creamos una función par el modelo
    def model(self, x):
        # input: [1,0,1,0][x1,x2,...,xn]
        z = sum(x)
        return (z >= self.threshold)

    # Creamos la predicción
    def predict(self, X):
        # input: [[1,0,1,0],[1,0,1,1]]
        Y = []
        for x in X:
            result = self.model(x)
            Y.append(result)
        return np.array(Y)

    # función de entrenamiento para el threshold
    def fit(self, X, Y):
        accuracy = {}

        # Seleccionamos un threshold entre el # de características de entrada
        for th in range(X.shape[1] + 1):
            self.threshold = th
            Y_pred = self.predict(X)
            accuracy[th] = accuracy_score(Y_pred, Y)

        # Seleccionamos el threshold que mejores resultado proporciona
        self.threshold = max(accuracy, key=accuracy.get)
```

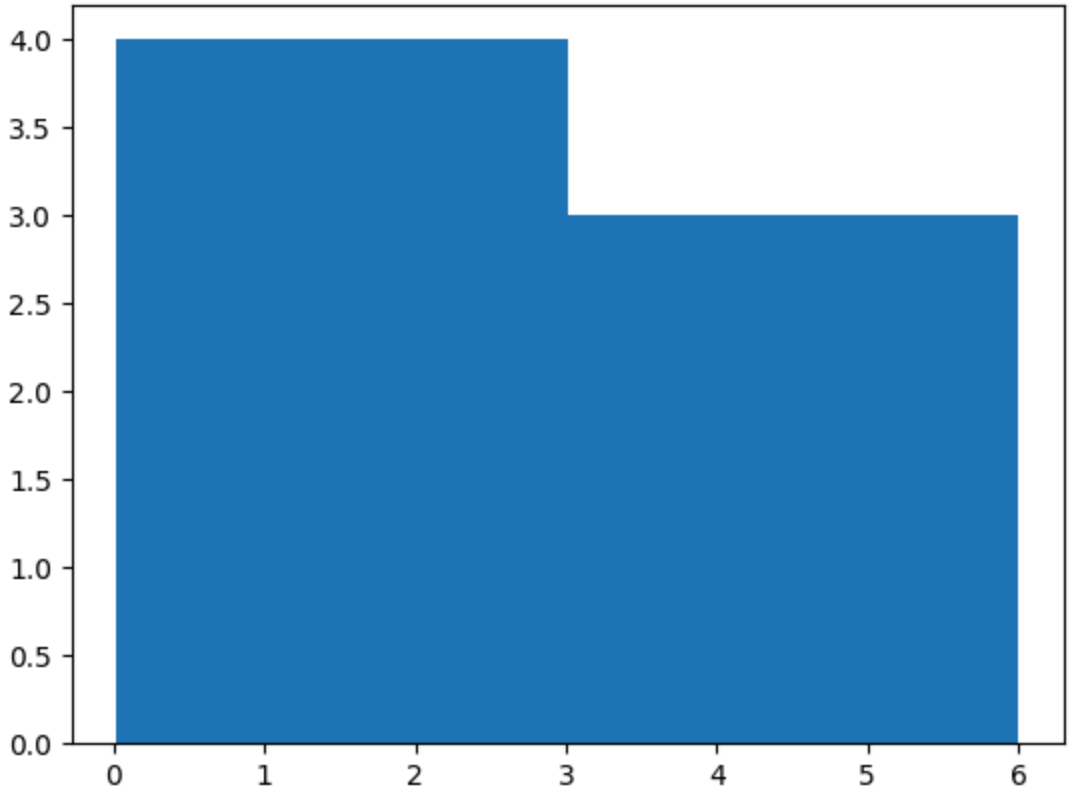
Los datos de entrada siguen teniendo problemas debido a que en nuestro conjunto de datos las características de entrada reciben valores continuos; sin embargo, la neurona MP solo procesa características de entrada con valores binarios.

In [8]:
```python
# Para transformar un valor a binario
import matplotlib.pyplot as plt

print(pd.cut([0.04, 0.3, 4, 5, 6, 0.02, 0.6], bins=2, labels=[0,1]))

plt.hist([0.04, 0.3, 4, 5, 6, 0.02, 0.6], bins=2)
plt.show()
```

```
[0, 0, 1, 1, 1, 0, 0]
Categories (2, int64): [0 < 1]
```



In [9]:
```python
# Transformarmamos las características de entrada a un valor binario
X_train_bin = X_train.apply(pd.cut, bins=2, labels=[1,0])
X_test_bin = X_test.apply(pd.cut, bins=2, labels=[1,0])

X_train_bin
```

Out[9]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	syr
178	1	0	1	1	1	1	1	1	
145	1	1	1	1	0	1	1	1	
204	1	1	1	1	1	1	1	1	
29	0	1	1	1	1	1	1	1	
416	1	1	1	1	1	1	1	1	
...	...	...	...	...	...	...	...	...	
252	0	1	0	1	1	0	0	1	
192	1	1	1	1	1	1	1	1	
27	0	1	0	1	1	1	1	1	
57	1	1	1	1	0	1	1	1	
424	1	1	1	1	1	1	1	1	

426 rows × 30 columns

In [10]:

```
# instaciamos el modelo MP
np_neurona = MpNeurona()

# Encontramos el threshold optimo
np_neurona.fit(X_train_bin.to_numpy(), y_train)

# threshold optimo seleccionado
np_neurona.threshold
```

Out[10]: 28

In [11]:

```
# Realizamos la predicción par alos datos de prueba
Y_pred = np_neurona.predict(X_test_bin.to_numpy())
```

In [12]:

```
Y_pred
```

Out[12]:
```
array([False,  True, False,  True,  True,  True, False,  True, False,
       False,  True, False,  True,  True,  True, False, False, False,
        True,  True,  True, False, False, False, False, False,  True,
       False,  True,  True,  True,  True,  True, False, False,  True,
       False, False, False, False, False,  True,  True, False,  True,
        True, False, False,  True,  True,  True, False, False, False,
       False,  True, False,  True,  True, False,  True, False,  True,
       False,  True,  True,  True, False,  True,  True, False,  True,
        True, False,  True, False, False, False,  True, False, False,
        True, False,  True,  True, False,  True,  True, False, False,
       False, False,  True, False, False, False,  True,  True,  True,
       False,  True,  True,  True,  True, False, False, False, False,
       False,  True, False,  True,  True, False, False,  True,  True,
        True, False,  True,  True,  True,  True, False, False,  True,
        True,  True,  True,  True, False, False,  True,  True,  True,
       False, False,  True,  True, False, False, False,  True])
```

In [13]:
```python
# Calculaculamos la exactitud de nuestra predicción
accuracy_score(y_test, Y_pred)
```

Out[13]:
```
0.8181818181818182
```

5. Github

https://github.com/Erick-305/machine-learning.git